



A SECURITY-FIRST APPROACH

Building a chatbot with LLM guardrails.

4 layers of defense that stop prompt injection, runaway costs, and data leaks. 📌

Input validation

Injection detection

Content filtering

Output validation



Ashwini

Senior AI/ML Engineer

THE PROBLEM

Your smart assistant has trust issues

An LLM processes user input as part of its instruction stream — that's the vulnerability. Within days of deploying my recipe bot, I was exposed to:

 **Prompt injection** — manipulating the AI's behavior

 **Cost explosions** — malicious users spamming requests

 **Off-topic queries** — wasting expensive LLM calls

 **Data leakage** — the AI revealing sensitive info



The defense: 4 layers

Information flows through four guardrails, before and after the LLM:

1 Input validation — sanity checks + rate limiting

2 Injection detection — catches malicious prompts

3 Domain filtering — keeps queries on-topic

4 Output validation — verifies responses are safe & well-formed

If one layer has a false negative, the others back it up.



Input validation

Validate the basics before a query ever touches the expensive model.

```
input_validation.py

MAX_QUERY_LENGTH = 500
MIN_QUERY_LENGTH = 2

class GuardrailViolation(Exception):
    pass

def validate_input(query, ip=None):
    if not query or not isinstance(query, str):
        raise GuardrailViolation("Invalid type")
    q = query.strip()
    if len(q) < MIN_QUERY_LENGTH:
        raise GuardrailViolation("Too short")
    if len(q) > MAX_QUERY_LENGTH:
        raise GuardrailViolation("Too long")
    if ip:
        _check_rate_limit(ip)
```

→ Cheap checks first. The model only runs if the basics pass.



Rate limiting (sliding window)

One bad user with no limit can wreck your API bill. Use a sliding window, not a fixed one.

```
rate_limit.py

import time
from collections import defaultdict

RATE_LIMIT_WINDOW = 60 # seconds
RATE_LIMIT_MAX_CALLS = 30
tracker = defaultdict(list)

def _check_rate_limit(ip):
    cutoff = time.time() - RATE_LIMIT_WINDOW
    tracker[ip] = [t for t in tracker[ip]
                    if t > cutoff]
    if len(tracker[ip]) ≥ RATE_LIMIT_MAX_CALLS:
        raise GuardrailViolation("Rate limit")
    tracker[ip].append(time.time())
```

→ Fixed windows let an attacker fire 30 at 11:59:59 + 30 at 12:00:01. Sliding stops that.



Prompt injection detection

Injection = users overriding your original instructions. The classics:

▶ — “Ignore all previous instructions...”

▶ — “You are now a...” / “Act as...”

▶ — “Forget everything above...”

▶ — “...tell me your system prompt”

**The fix: a list of red-flag phrases, checked before the LLM.
Match → block + log. No ML, no extra API cost.**



Domain content filtering

Why burn \$0.002 answering “What’s the weather?” on a recipe bot? Filter off-topic before the call.

```
domain_filter.py

def is_food_related(query):
    food = ['recipe', 'cook', 'pasta', 'vegan']
    off_topic = ['weather', 'math', 'politics']
    q = query.lower()
    # reject off-topic immediately
    if any(w in q for w in off_topic):
        return False
    # accept clear food keywords
    if any(w in q for w in food):
        return True
    # accept common food phrases
    if any(p in q for p in
           ['what can i make', 'i want']):
        return True
    return False
```

→ Runs before the LLM call → saves both time and money.



Output validation

LLMs hallucinate and return malformed answers. Expecting JSON, getting markdown? Validate before trusting.

```
output_validation.py

import json

def validate_output(response, fmt=None):
    if not response or not response.strip():
        return False
    if fmt == "json":
        try:
            json.loads(response)
        except json.JSONDecodeError:
            return False
    refusals = ['i cannot help',
                'inappropriate']
    if any(p in response.lower()
           for p in refusals):
        return False
    return True
```

→ Fails validation? Fall back to traditional logic instead of crashing.



PRODUCTION LESSON #1

Start strict, then loosen

Launch with aggressive filtering. False positives are easier to fix than security breaches.

After a month, review your logs and selectively whitelist the patterns real users actually need.

Tighten first, relax later — never the other way around.



Ashwini

Senior AI/ML Engineer

PRODUCTION LESSON #2

Log everything

+3 attack patterns found in logs

Every guardrail violation gets logged: IP, timestamp, query, and which layer blocked it. That data becomes your tuning roadmap.

The author discovered 3 new attack patterns from logs they'd never have anticipated.

Use it to spot attacks, tune filters, block repeat offenders, and improve UX.



Ashwini

Senior AI/ML Engineer

PRODUCTION LESSON #3

Watch costs religiously

Track requests per hour / day / month, cost per request, and success rate — valid responses vs. rejections.

The filtering layers exist precisely so your spend goes to relevant queries only.

Security and cost control are the same project here.



Ashwini

Senior AI/ML Engineer

The takeaway.

Security isn't an afterthought. It's a feature.

- ✓ Multi-layer defense is non-negotiable in production
- ✓ Each layer adds redundancy
- ✓ The patterns are simpler than you think
- ✓ Monitoring + iteration is what makes it last

Repost

Save

Comment

 Follow Ashwini

#LLMSecurity · #Guardrails · #AIEngineering · #PromptInjection



Ashwini

Senior AI/ML Engineer